



COMANDOS PYTHON - GUIA COMPLETO



CONFIGURAÇÃO E AMBIENTES

Instalação e Versionamento

```
bash
```

```
# Verificar versão do Python
```

```
python --version
```

```
python3 --version
```

```
# Instalar Python no Ubuntu/Debian
```

```
sudo apt update
```

```
sudo apt install python3 python3-pip python3-venv
```

```
# Instalar Python no macOS
```

```
brew install python
```

```
# Instalar Python no Windows
```

```
# Download: https://python.org/downloads/
```

```
# Instalar múltiplas versões (pyenv)
```

```
curl https://pyenv.run | bash
```

```
pyenv install 3.11.0
```

```
pyenv global 3.11.0
```

Ambientes Virtuais

```
bash
```

```
# Criar ambiente virtual
```

```
python3 -m venv meu_ambiente
```

```
# ou
```

```
virtualenv meu_ambiente
```

```
# Ativar ambiente (Linux/macOS)
```

```
source meu_ambiente/bin/activate
```

```
# Ativar ambiente (Windows)
meu_ambiente\Scripts\activate
```

```
# Desativar ambiente
deactivate
```

```
# Requirements.txt
pip freeze > requirements.txt
```

```
pip install -r requirements.txt
```



GESTÃO DE PACOTES COM PIP

Comandos Básicos do PIP

```
bash
```

```
# Instalar pacote
pip install requests
pip install django==4.2.0
pip install "package>=2.0,<3.0"
```

```
# Listar pacotes instalados
pip list
pip freeze
```

```
# Mostrar informações do pacote
pip show requests
```

```
# Verificar pacotes desatualizados
pip list --outdated
```

```
# Atualizar pacote
pip install --upgrade requests
```

```
# Desinstalar pacote
```

```
pip uninstall requests
```

PIP Avançado

```
bash
```

```
# Instalar de repositório Git
```

```
pip install git+https://github.com/usuario/repositorio.git
```

```
pip install git+https://github.com/usuario/repositorio.git@branch
```

```
# Instalar em modo desenvolvimento
```

```
pip install -e .
```

```
# Instalar com dependências extras
```

```
pip install "package[dev,test]"
```

```
# Configurar mirror (China)
```

```
pip install -i https://pypi.tuna.tsinghua.edu.cn/simple/ package
```

```
# Cache do pip
```

```
pip install --no-cache-dir package
```



ESTRUTURA DE PROJETOS

Criando Projetos

```
bash
```

```
# Estrutura básica de projeto
```

```
mkdir meu_projeto
```

```
cd meu_projeto
```

```
python -m venv venv
```

```
source venv/bin/activate
```

```
# Criar estrutura de pastas
```

```
mkdir -p src/tests docs
```

```
touch src/__init__.py
```

```
touch src/main.py
```

```
touch requirements.txt
```

```
touch README.md
```

Exemplo de Estrutura

```
text
```

```
meu_projeto/
├── src/
│   ├── __init__.py
│   ├── main.py
│   ├── utils/
│   │   ├── __init__.py
│   │   └── helpers.py
│   └── models/
│       ├── __init__.py
│       └── user.py
├── tests/
│   ├── __init__.py
│   ├── test_main.py
│   └── test_utils/
├── docs/
├── requirements.txt
├── setup.py
└── README.md
```



COMANDOS DE DESENVOLVIMENTO

Execução de Scripts

```
bash
```

```
# Executar script Python
```

```
python script.py
```

```
python3 script.py
```

```
# Executar com arguments
```

```
python script.py arg1 arg2
```

```
# Executar módulo como script
```

```
python -m module_name
```

```
# Executar com variáveis de ambiente
```

```
DEBUG=true python script.py
```

Debug e Depuração

```
bash
```

```
# Debug com pdb
```

```
python -m pdb script.py
```

```
# Executar com otimizações
```

```
python -O script.py # Remove asserts
```

```
python -OO script.py # Remove asserts e docstrings
```

```
# Profiling
```

```
python -m cProfile script.py
```

```
# Verificar sintaxe
```

```
python -m py_compile script.py
```

```
python -m compileall .
```

IPython e Jupyter

```
bash
```

```
# Instalar e usar IPython
```

```
pip install ipython
```

```
ipython
```

```
# Jupyter Notebook
```

```
pip install jupyter
```

```
jupyter notebook
```

```
# Jupyter Lab
```

```
pip install jupyterlab
```

```
jupyter lab
```

```
# Exportar notebook
```

```
jupyter nbconvert --to pdf notebook.ipynb
```



TESTES E QUALIDADE DE CÓDIGO

Frameworks de Teste

```
bash
```

```
# unittest (built-in)
python -m unittest discover
python -m unittest test_module.py
python -m unittest test_module.TestClass.test_method
```

```
# pytest
pip install pytest
pytest
pytest -v # verbose
pytest -x # para no primeiro erro
pytest --cov=src # coverage
```

```
# nose2
pip install nose2
```

```
nose2
```

Quality Assurance

```
bash
```

```
# Flake8 (linter)
pip install flake8
flake8 src/
```

```
# Black (formatter)
pip install black
black src/
```

```
# isort (organizar imports)
pip install isort
isort src/
```

```
# mypy (type checking)
pip install mypy
mypy src/
```

```
# Bandit (security)
pip install bandit
bandit -r src/
```

```
# Safety (vulnerabilities)
pip install safety
```

```
safety check
```

Coverage

```
bash
```

```
# Coverage.py
pip install coverage
coverage run -m pytest
coverage report
coverage html # gera relatório HTML
```

```
# Com pytest-cov
```

```
pytest --cov=src --cov-report=html
```



COMANDOS PARA CI/CD

Setup.py e Packaging

```
python
```

```
# setup.py exemplo
from setuptools import setup, find_packages
```

```
setup(
    name="meu-pacote",
    version="0.1.0",
    packages=find_packages(),
    install_requires=[
        "requests>=2.25.0",
        "click>=8.0.0",
    ],
    entry_points={
        'console_scripts': [
            'meu-comando=src.main:main',
        ]
    }
)
```

```
    ],  
  },  
)  
)
```

Comandos de Build

```
bash
```

```
# Build do pacote  
python setup.py sdist bdist_wheel
```

```
# Instalar em desenvolvimento  
pip install -e .
```

```
# Twine (upload para PyPI)  
pip install twine  
twine upload dist/*
```

```
# Verificar package
```

```
twine check dist/*
```



PYTHON COM DOCKER

Dockerfile para Python

```
dockerfile
```

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
# Instalar dependências do sistema  
RUN apt-get update && apt-get install -y \  
    gcc \  
    && rm -rf /var/lib/apt/lists/*
```

```
# Copiar requirements e instalar dependências  
COPY requirements.txt .
```



```
RUN pip install --no-cache-dir -r requirements.txt
```

```
# Copiar código
```

```
COPY . .
```

```
# Comando para executar
```

```
CMD ["python", "app.py"]
```

Docker Compose para Desenvolvimento

```
yaml
```

```
version: '3.8'
```

```
services:
```

```
  web:
```

```
    build: .
```

```
    ports:
```

```
      - "8000:8000"
```

```
    volumes:
```

```
      - ./app
```

```
    environment:
```

```
      - DEBUG=true
```

```
    command: python manage.py runserver 0.0.0.0:8000
```

```
  redis:
```

```
    image: redis:alpine
```

```
  db:
```

```
    image: postgres:13
```

```
    environment:
```

```
      POSTGRES_DB: myapp
```

```
      POSTGRES_USER: user
```

```
      POSTGRES_PASSWORD: pass
```



PYTHON COM CLOUD

AWS (Boto3)

```
bash
```

```
# Instalar boto3  
pip install boto3
```

```
# Configurar credenciais  
aws configure
```

```
# Exemplo de uso
```

```
python -c "import boto3; print(boto3.client('s3').list_buckets())"
```

Azure (Azure SDK)

```
bash
```

```
# Instalar Azure SDK  
pip install azure-identity azure-storage-blob
```

```
# Autenticar
```

```
az login
```

Google Cloud

```
bash
```

```
# Instalar Google Cloud SDK  
pip install google-cloud-storage
```

```
# Autenticar
```

```
gcloud auth application-default login
```



COMANDOS PARA ANÁLISE DE DADOS

Instalação de Bibliotecas de Data Science

```
bash
```

```
# Instalar stack básica
```

```
pip install numpy pandas matplotlib seaborn jupyter
```

```
# Instalar stack completa
```

```
pip install numpy pandas matplotlib seaborn plotly scipy scikit-learn jupyter
```

```
# Instalar com conda
```

```
conda install numpy pandas matplotlib seaborn
```

Comandos Úteis para Análise

```
bash
```

```
# Executar análise em lote
```

```
python analise_dados.py
```

```
# Gerar relatórios automáticos
```

```
python -c "import pandas as pd; df=pd.read_csv('dados.csv'); print(df.describe())"
```

```
# Profile de dados com pandas-profiling
```

```
pip install pandas-profiling
```

```
python -c "from pandas_profiling import ProfileReport;
ProfileReport(pd.read_csv('dados.csv')).to_file('relatorio.html')"
```



FRAMEWORKS POPULARES

Django

```
bash
```

```
# Instalação e comandos
```

```
pip install django
```

```
django-admin startproject meu_projeto
```

```
cd meu_projeto
```

```
python manage.py startapp minha_app
```

```
python manage.py runserver
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
```

```
python manage.py shell
```

Flask

```
bash
```

Instalação e execução

```
pip install flask
```

```
python app.py
```

ou

```
flask run
```

Com variáveis de ambiente

```
export FLASK_APP=app.py
```

```
export FLASK_ENV=development
```

```
flask run
```

FastAPI

```
bash
```

Instalação e execução

```
pip install fastapi uvicorn
```

```
uvicorn main:app --reload
```

Com diferentes hosts/portas

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

Production

```
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```



FERRAMENTAS DE DESENVOLVIMENTO

Pre-commit Hooks

```
bash
```

```
# Instalar pre-commit
pip install pre-commit

# Configurar .pre-commit-config.yaml
pre-commit install
```

```
pre-commit run --all-files
```

Poetry (Gerenciamento de Dependências)

```
bash
```

```
# Instalar Poetry
curl -sSL https://install.python-poetry.org | python3 -

# Comandos básicos
poetry new meu_projeto
poetry install
poetry add requests
poetry add --dev pytest
poetry run python script.py
```

```
poetry build
```

Debugging Avançado

```
bash
```

```
# pdb++ (pdb melhorado)
pip install pdbpp
python -m pdb script.py

# ipdb (pdb com IPython)
pip install ipdb
import ipdb; ipdb.set_trace()
```

Debug com VSCode

```
# Criar .vscode/launch.json
```



SCRIPTS ÚTEIS

Script de Desenvolvimento

```
bash
```

```
#!/bin/bash
```

```
# dev.sh
```

```
echo "🐍 Iniciando ambiente de desenvolvimento Python"
```

```
# Criar ambiente virtual se não existir
```

```
if [ ! -d "venv" ]; then
```

```
    echo "Criando ambiente virtual..."
```

```
    python3 -m venv venv
```

```
fi
```

```
# Ativar ambiente
```

```
source venv/bin/activate
```

```
# Instalar/atualizar dependências
```

```
echo "Instalando dependências..."
```

```
pip install -r requirements.txt
```

```
# Executar testes
```

```
echo "Executando testes..."
```

```
pytest
```

```
# Executar linter
```

```
echo "Verificando qualidade do código..."
```

```
flake8 src/
```

```
echo "✅ Ambiente pronto!"
```

Script de Deploy

```
bash
```

```
#!/bin/bash
# deploy.sh

set -e # Para em caso de erro

echo "🚀 Iniciando deploy..."

# Ativar ambiente virtual
source venv/bin/activate

# Instalar dependências
pip install -r requirements.txt

# Executar migrações (se for Django)
python manage.py migrate

# Coletar arquivos estáticos (se for Django)
python manage.py collectstatic --noinput

# Executar testes
pytest
```

```
echo "✅ Deploy concluído com sucesso!"
```

Script de Backup de Dados

```
python
```

```
#!/usr/bin/env python3
# backup.py

import sqlite3
import pandas as pd
import json
from datetime import datetime
import subprocess

def backup_database():
    """Faz backup do banco de dados"""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    # Backup SQLite
    conn = sqlite3.connect('database.db')
```

```

# Exportar tabelas para CSV
tables = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table'",
conn)

for table in tables['name']:
    df = pd.read_sql(f"SELECT * FROM {table}", conn)
    df.to_csv(f'backup/{table}_{timestamp}.csv', index=False)

conn.close()
print(f"Backup concluído: {timestamp}")

if __name__ == "__main__":
    backup_database()

```

TROUBLESHOOTING

Problemas Comuns

```
bash
```

```

# Problema: ModuleNotFoundError
# Solução: Verificar PYTHONPATH ou instalar pacote
export PYTHONPATH="${PYTHONPATH}:/caminho/para/seu/projeto"
pip install nome_do_pacote

```

```

# Problema: Permissão negada
# Solução: Usar venv ou --user
pip install --user package

```

```

# Problema: Versão conflitante
# Solução: Usar venv ou pipenv
python -m venv myenv
source myenv/bin/activate

```

```

# Problema: SSL Certificate verify failed
# Solução: Configurar certificado ou usar --trusted-host

```

```

pip install --trusted-host pypi.org --trusted-host pypi.python.org
--trusted-host files.pythonhosted.org package

```

Performance


```
bash
```

```
# Otimizar imports
```

```
python -O -m py_compile script.py # Gera bytecode otimizado
```

```
# Medir tempo de execução
```

```
time python script.py
```

```
# Memory profiling
```

```
pip install memory_profiler
```

```
python -m memory_profiler script.py
```

Este guia cobre todos os comandos essenciais para desenvolvimento Python profissional! 🐍